

3. 数据类型

1. 本节目标

- 掌握数据类型分类
- 掌握常用数据类型

2. 常用数据类型分类

在面向对象软件开发的过程中，通常会先进行需求分析从而得到**类**和**属性**，类是面向对象中的概念，对应到数据库中的概念就是**实体**，类中的属性对应实体中的**属性**。实体通常以**表**的形式存在，每个实体对应一张表，表中的每条记录(数据行)就是实体的一个实例，每条记录又包含若干**字段**(或称为**列**)，每个字段代表实体的一个属性。

如果要定义实体的属性，就要为属性命名并指定合适的数据类型。与其他编程语言类似，SQL中规定了用于描述属性的数据类型。常用的数据类型有以下几类：

- 数据值类型
- 字符串类型
- 二进制类型
- 日期类型

3. 数据值类型

3.1 类型列表

类型	大小	说明
BIT[(M)]	默认 bit	位值类型。M表示每个值的位数，取值范围为 1 ~ 64 。如果省略M，则默认为 1 。
TINYINT[(M)]	1byte	取值范围 $-2^7 \sim 2^7-1$ ，无符号取值范围 2^8-1
BOOL	1byte	TINYINT(1)的同义词。值为零被认为是假，非零值被认为是true
SMALLINT[(M)]	2byte	取值范围 $-2^{15} \sim 2^{15}-1$ ，无符号取值范围 $2^{16}-1$ 。
MEDIUMINT[(M)]	3byte	取值范围 $-2^{23} \sim 2^{23}-1$ ，无符号取值范围 $2^{24}-1$
INT[(M)]	4byte	取值范围 $-2^{31} \sim 2^{31}-1$ ，无符号取值范围 $2^{32}-1$

INTEGER[(M)]	4byte	INT[(M)]的同义词
BIGINT[(M)]	8byte	取值范围 $-2^{63} \sim 2^{63}-1$ ，无符号取值范围 $2^{64}-1$
FLOAT[(M,D)]	4byte	单精度浮点型，M是总位数，D是小数点后面的位数，大约可以精确到小数点后7位
DOUBLE[(M,D)]	8byte	双精度浮点型，M是总位数，D是小数点后面的位数，大约可以精确到小数点后15位。
DECIMAL[(M[,D])]	动态	不存在精度损失，M是总位数，D是小数点后的位数。DECIMAL的最大位数(M)为65，最大小数位数(D)为30。如果省略M，则默认为10，如果省略D，则默认为0。M中不计算小数点和负数的-号，如果D为0，则值没有小数点和小数部分。

3.2 数据类型取值范围

类型	大小	有符号最小值	有符号最大值	无符号最小值	无符号最大值
TINYINT	1Byte	-128	127	0	255
SMALLINT	2Byte	-32768	32767	0	65535
MEDIUMINT	3Byte	-8388608	8388607	0	16777215
INT	4Byte	-2147483648	2147483647	0	4294967295
BIGINT	8Byte	-2^{63}	$2^{63}-1$	0	$2^{64}-1$

4. 字符串类型

4.1 类型列表

类型	说明
CHAR[(M)]	固定长度字符串，M表示长度，以字符为单位，取值范围 $0 \sim 255$ ，M省略则长度为 1
VARCHAR(M)	

	可变长度字符串， M 表示字符最大长度，取值范围 <code>0 ~ 65535</code> ，有效字符个数取决于实际字符数和使用的字符集
TINYTEXT	小文本类型，最大长度为 <code>255</code> ($2^8 - 1$)个字节，有效字符个数取决于使用的字符集
TEXT[(M)]	文本类型，最大长度为 <code>65535</code> ($2^{16} - 1$)个字节，有效字符个数取决于使用的字符集
MEDIUMTEXT	中文本类型，最大长度为 <code>16,777,215</code> ($2^{24} - 1$)个字节，有效字符个数取决于使用的字符集
LONGTEXT	大文本类型，最大长度为 <code>4,294,967,295</code> 即 <code>4GB</code> ($2^{32} - 1$)个字节，有效字符个数取决于使用的字符集
BINARY[(M)]	固定长度二进制字节，于CHAR类似，但存储的是二进制字节而不是字符串。 M 表示长度，以 字节 为单位，取值范围 <code>0 ~ 255</code> ， M 省略则长度为 <code>1</code>
VARBINARY(M)	可变长度二进制字节，于VARCHAR类似，但存储的是二进制字节而不是字符串。 M 表示长度，以 字节 为单位
TINYBLOB	小二进制字节类型，最大长度为 <code>255</code> ($2^8 - 1$)个字节
BLOB[(M)]	二进制字节类型，最大长度为 <code>65535</code> ($2^{16} - 1$)个字节
MEDIUMBLOB	中二进制字节类型，最大长度为 <code>16,777,215</code> ($2^{24} - 1$)个字节
LOBLOB	大二进制字节类型，最大长度为 <code>4,294,967,295</code> 即 <code>4GB</code> ($2^8 - 1$)个字节
ENUM('value1','value2',...)	- 枚举 - 从值列表 <code>'value1','value2'</code> 或 <code>''</code> (空字符串) 和 <code>NULL</code> 中选一个值 - 最多可以有 <code>65,535</code> 个不同的元素 - 单个元素的最大长度是 <code>M ≤ 255</code> 或 <code>(M × w) ≤ 1020</code>，其中 M 是元素字符长度，w 是字符集中字符所需的最大字节数 - ENUM的值在内部表示为整数
SET('value1','value2',...)	- 集合 - 从值列表 <code>'value1','value2'</code> 中选零个或多个值 - 最多64个元素 - 单个元素的最大长度是 <code>M ≤ 255</code> 或 <code>(M × w) ≤ 1020</code>，其中 M 是元素字符长度，w 是字符集中字符所需的最大字节数 - SET值在内部表示为整数

4.2 关于排序

- 字符串类型的列以字符为单位，并且可以单独指定字符集和排序规则，比如字符集是 `utf8mb4`，排序规则是 `utf8_0900_ai_ci`
- 二进制的列以字节为单位，可以指定_bin结尾的排序规则，比如排序规则是 `utf8mb4_bin`，这时以比较和排序基于数字字符代码值

4.3 CHAR与VARCHAR的区别

- **CHAR** 固定长度的字符串，**M** 表示以字符为单位的列长度，取值范围 `0 ~ 255`，省略则长度为 `1`，在存储时总是用空格向右填充到指定的长度，获取列的值时会从尾部删除空格。允许定义 `CHAR(0)`，此时列的值只能为NULL或空字符串，主要的目的是为了旧系兼容，比如类中必须有这个属性，但不使用这个属性的值，也就是说值并没有意义，但列又不能没有。
- **VARCHAR** 可变长度字符串。**M** 表示以字符为单位的最大列长度，取值范围 `0 ~ 65,535` (在所有列中共享)，有效长度取决于实际字符数和使用的字符集，并且用额外的一或两个字节记录实际使用的字节数，当实际字节数不超过 255 个字节用一个字节记录长度，超过 255 个字节时，使用两个字节记录长度，获取列的值时不会从尾部删除空格，插入数据时会删除超出长度的空格。

```

1 mysql> CREATE TABLE vc (v VARCHAR(4), c CHAR(4));
2 Query OK, 0 rows affected (0.01 sec)
3
4 mysql> INSERT INTO vc VALUES ('ab ', 'ab ');
5 Query OK, 1 row affected (0.00 sec)
6
7 mysql> SELECT CONCAT('(', v, ')'), CONCAT('(', c, ')') FROM vc;
8 +-----+-----+
9 | CONCAT('(', v, ')') | CONCAT('(', c, ')') |
10 +-----+-----+
11 | (ab )           | (ab)                |
12 +-----+-----+
13 1 row in set (0.06 sec)
14
15 mysql> insert into vc values ('ab ', 'ab ');
16 Query OK, 1 row affected, 1 warning (0.01 sec)
17
18 mysql> show warnings;
19 +-----+-----+-----+
20 | Level | Code | Message                                     |
21 +-----+-----+-----+
22 | Note  | 1265 | Data truncated for column 'v' at row 1 |
23 +-----+-----+-----+
24 1 row in set (0.00 sec)

```

- 通过下表来说明CHAR和VARCHAR存储字符串的区别，字符集以latin1为例：

值	CHAR(4)	所需要空间大小	VARCHAR(4)	所需要空间大小
"	' '	4 bytes	"	1 byte
'ab'	'ab '	4 bytes	'ab'	3 bytes
'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
'abcdefgh'	X	超出最大长度	X	超出最大长度

4.4 如何选择CHAR与VARCHAR

- 如果数据确定长度都一样，就使用定长 CHAR 类型，比如：身份证，md5，学号，邮编。
- 如果数据长度有变化,就使用变长 VARCHAR ,比如：名字，地址，但要规划好长度，保证最长的字符串能存的进去。
- 定长 CHAR 类型比较浪费磁盘空间，但是效率高。
- 变长 VARCHAR 类型比较节省磁盘空间，但是效率低。
- 定长 CHAR 类型会直接开辟好对应的存储空间。
- 变长 VARCHAR 类型在不超过定义长度范围的情况下用多少开辟多少存储空间。

4.5 VARCHAR与TEXT的区别

- **容量大小：** VARCHAR 最大支持 65535 个字节； TEXT 最大支持 65535 个字节，在指定TEXT长度时，当超过 65535 时自动转换为 MEDIUMTEXT 类型，当超过 16,777,215 时自动转换为 LONGTEXT 类型
- **存储位置：** VARCHAR 类型的列实际内容小于768个字节时存在当前行，大于768时存在溢出页，当前行保存溢出页的地址； TEXT 类型的列整体保存在溢出页，当前行只保存溢出页地址
- **查询性能：** 对于频繁查询的 VARCHAR 列可以创建索引，提升查询性能； TEXT 类型的列无法直接创建普通索引，但可以使用 FULLTEXT 索引，由于索引的支持和存储位置的不同， VARCHAR 列的性能高于 TEXT 类型的列
- **适用场景：** 如果存储的数据长度较小且需要创建索引进行检索，可以选择 VARCHAR 类型，比如姓名，用户，邮箱等；如果存储的数据长度较大且不需要频繁以该列为条件进行检索可以选择 TEXT 类型，比如文章内容等。

5. 日期类型

5.1 类型列表

类型	大小	说明	0值
		• 时间戳类型	

<code>TIMESTAMP[(fsp)]</code>	4 bytes	- 支持范围 1970-01-01 00:00:01.000000 ~ 2038-01-19 03:14:07.499999 0值为 0000-00-00 00:00:00	0000-00-00 00:00:00
<code>DATETIME[(fsp)]</code>	8 bytes	- 日期类型和时间类型的组合 - 支持范围 1000-01-01 00:00:00.000000 ~ 9999-12-31 23:59:59.499999 - 显示格式为 YYYY-MM-DD hh:mm:ss[.fraction] 0值为 0000-00-00 00:00:00	0000-00-00 00:00:00
<code>DATE</code>	3 bytes	- 日期类型 - 支持范围 1000-01-01 ~ 9999-12-31 - 显示格式为 YYYY-MM-DD 0值为 0000-00-00	
<code>TIME[(fsp)]</code>	3 bytes	- 时间类型 - 支持范围 -838:59:59.000000 ~ 838:59:59.000000 - 显示格式为 hh:mm:ss[.fraction] 0值为 00:00:00	00:00:00
<code>YEAR[(4)]</code>	1 byte	4位格式的年份 - 支持范围 1901 ~ 2155 - 显示格式为 YYYY 0值为 0	0

5.2 其他

- `fsp` 为可选设置，用来指定小数秒精度，范围从0到6，值为0表示没有小数部分，如果省略，默认精度为0
- `CURRENT_DATE` 和 `CURRENT_DATE()` 是 `CURDATE()` 的同义词用于获取当前日期
- `CURRENT_TIME` 和 `CURRENT_TIME([fsp])` 是 `CURTIME()` 的同义词用于获取当前时间
- `CURRENT_TIMESTAMP` 和 `CURRENT_TIMESTAMP([fsp])` 是 `NOW()` 的同义词用于获取当前日期和时间